

01

Outline

인사 함수의 자동 검사

- 이름을 넣으면 인사말을 반환하는 함수를 검사한다. 입력·기대값을 코드로 적고 pytest로 실행한다.
- 소스 수정 반영, 테스트 발견, 실패 보고서를 확인한다.

테스트의 반복과 빠진 검사

- 같은 준비를 반복하는 방법과 여러 입력을 검사하는 방법을 구분한다.
- 테스트끼리 상태를 공유할 때의 실패를 고친다. 실행한 줄의 비율과 입력별 정확성을 구분한다.

02

검증 대상과 공개 계약

src/hello_open_source/greeting.py

- 사용자는 이름을 넣어 인사말을 받는다. 함수 `greet` 의 계약은 정확한 문자열 반환이다.
- `name: str -> str` 는 형식 표기다. `f"...{name}..."` 의 중괄호에 이름이 들어간다.

```
def greet(name: str) -> str:  
    return f"Hello, {name}!"
```

```
greet("Hee") → "Hello, Hee!"  
greet("world") → "Hello, world!"
```

함수 검사와 명령 입력 검사

- `hello-greet` 는 터미널 명령이고 `main()` 은 인수를 읽어 출력과 정상 종료 값 0을 만드는 함수다.
- `main` 직접 호출은 인수→출력 연결을 검사한다. 설치된 실행 파일의 생성과 여러 Python 버전 호환성은 별도 검사다.

입력	main의 기대 결과
이름 Hee	Hello, Hee! 출력 / 0 반환
인수 없음	Hello, world! 출력 / 0 반환

03

명령 입력에서 출력까지

src/hello_open_source/cli.py 전체 코드

- CLI는 Command-Line Interface, 즉 터미널의 명령 입력 접점이다. `sys.argv`의 0번은 명령 이름, 1번부터 사용자 입력이다.
- `import sys`는 인수 목록을, `from ... import greet`는 인사 함수를 가져온다. 이름이 없으면 `world`를 선택한다.
- `print`는 문자열 뒤에 줄바꿈을 출력한다. `return 0`은 정상 종료 값이며 `-> int`는 정수 반환 표기다.

```
import sys
from hello_open_source.greeting import greet

def main() -> int:
    if len(sys.argv) > 1:
        name = sys.argv[1]
    else:
        name = "world"
    print(greet(name))
    return 0
```

04

첫 pytest 테스트

tests/test_greeting.py: 준비 → 실행 → 단언

- `pytest` 는 테스트를 찾아 실행·보고하는 도구다. 한 테스트는 입력·기대값·비교를 묶고, 스위트는 여러 테스트의 집합이다.
- `from ... import greet` 는 공개 함수를 가져온다. `"Hee"` 를 준비해 호출하고 결과를 `actual` 에 담는다.
- `assert` 는 비교가 참이면 통과, 거짓이면 실패다. `test_` 는 발견 규칙, `-> None` 은 반환값이 없다는 형식 표기다.

```
from hello_open_source.greeting import greet
```

```
def test_greet_with_name() -> None:  
    actual = greet("Hee")  
    assert actual == "Hello, Hee!"
```

실제 `"Hello, Hee!" == 기대 "Hello, Hee!"` → 통과
문자열이라는 사실만 검사하면 `"BROKEN"`도 통과할 수 있다.

05

편집 가능한 설치와 소스 수정

프로젝트 소스를 읽도록 한 번 설치

- 기존 `hello-open-source/` 에서 가상환경을 활성화한다. `.` 은 이 폴더, `-e` 는 이 폴더의 소스를 import할 수 있게 설치하는 옵션이다.
- 아래 명령의 `python -c` 는 따옴표 안 Python 코드를 새 프로세스에서 실행한다.

```
python -m pip install -e .
python -m pip install pytest pytest-cov
```

```
python -c "from hello_open_source.greeting import
greet; print(greet('Hee'))"
```

```
Hello, Hee!
```

함수 본문 수정 → 새 실행에서 바로 반영

- `src/hello_open_source/greeting.py` 의 반환문을 아래처럼 바꾸고 저장한다.
- 왼쪽 `python -c` 명령을 다시 실행하면 재설치 없이 바뀐 문자열이 나온다. 확인 후 `Hello` 로 되돌린다.

```
def greet(name: str) -> str:
    return f"Hi, {name}!"
```

```
Hi, Hee!
```

`python -m pytest tests/test_greeting.py -q` 로 첫 테스트를 실행하면 `1 passed` 다. `-q` 는 간결한 결과 표시다.

범위: 기존 Python 함수 본문 수정이다. 프로젝트의 설치 메타데이터·명령 등록을 바꾸면 재설치가 필요하다.

06

실패 진단과 회귀 검사

기대값이 계약과 다른 경우

- 계약과 실제값은 `Hello, Hee!` 인데 기대값을 `Hi, Hee!` 로 쓴 상황이다. 테스트를 고친다.
- `E` 는 오류 설명, `-` 는 오른쪽 기대값, `+` 는 왼쪽 실제값이다.

```
assert greet("Hee") == "Hi, Hee!"
```

```
E AssertionError
E   - Hi, Hee!
E   + Hello, Hee!
1 failed
```

구현이 계약과 다른 경우

- 회귀는 변경 때문에 이전 동작이 깨지는 현상이다. 구현에서 `!` 를 지웠다면 구현을 고친다.
- 정확한 기대값의 테스트를 유지한다. 같은 결함이 재등장하면 같은 입력으로 다시 실패한다.

```
계약: "Hello, Hee!"
```

```
구현에서 ! 삭제 → "Hello, Hee"
기존 테스트 실행 → 1 failed
```

```
구현의 ! 복구 → "Hello, Hee!"
같은 테스트 실행 → 1 passed
```

07

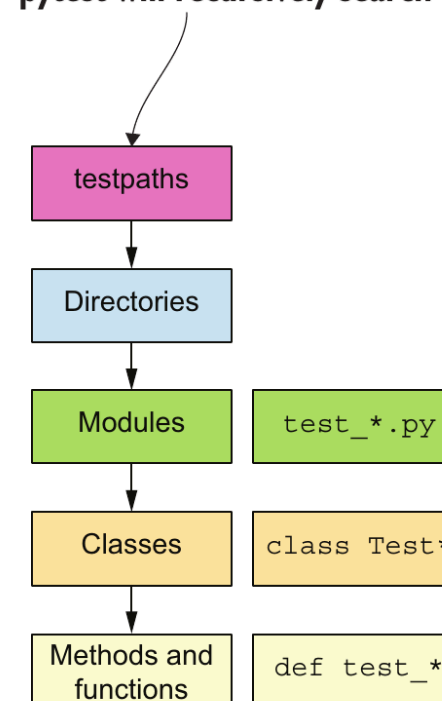
pytest의 테스트 탐색 구조

교재 Figure 5.1 읽기

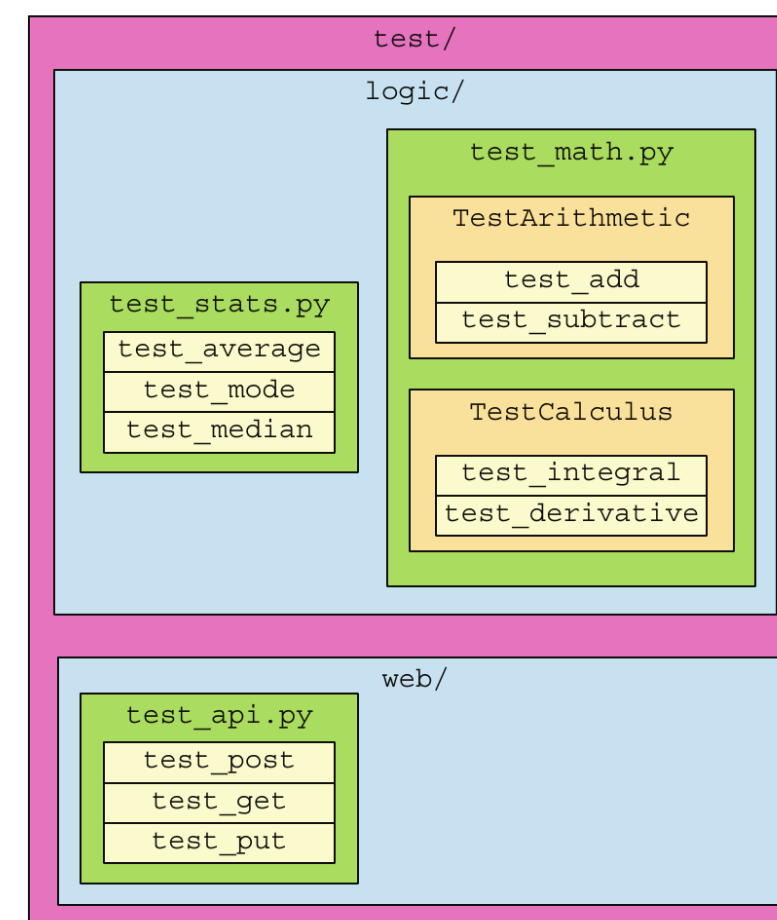
- 왼쪽 화살표는 탐색 순서다. `testpaths` 가 시작 위치를 정하고 폴더 → Python 파일 → 테스트 함수로 내려간다.
- 오른쪽 자홍색 `test/` 안에 파란 폴더 `logic/` · `web/` 가 있다. 초록색 상자는 `test_*.py` 파일이다.
- 노란 `Test*` 클래스 안의 함수도, 클래스 밖의 `test_*` 함수도 찾는다. 클래스는 필수가 아니다.

예: `test/logic/test_stats.py` 의 함수 3개 + `test_math.py` 의 함수 4개 + `test/web/test_api.py` 의 함수 3개 → 그림의 테스트 10개.

Starting in the directories defined by the configured `testpaths` value, pytest will recursively search for tests.



Some parts of the search look for specific name patterns, whose defaults you can override if desired.



With `testpaths` configured as `test`, `pytest` will find each of these tests by default.

Figure 5.1 pytest discovers the unit tests in a project using recursive pattern matching.

공유 상태와 테스트 순서

tests/test_state.py 전체 코드

- 두 테스트가 하나의 목록 FRUITS 를 함께 쓴다. `append` 는 그 목록에 원소를 추가하고 `len` 은 길이를 센다.

```
FRUITS = ["apple"]

def test_append():
    FRUITS.append("banana")
    assert FRUITS == ["apple", "banana"]

def test_len():
    assert len(FRUITS) == 1
```

```
python -m pytest tests/test_state.py -q
```

교재 Figure 5.5의 두 실행 순서

- 파란 상자는 목록 상태, 초록 상자는 검사다. 왼쪽은 길이 → 추가, 오른쪽은 추가 → 길이로 실행한다.
- 추가가 먼저면 목록이 2개로 바뀐다. 뒤의 길이 검사는 기대값 1과 달라 실패한다. 실행을 새로 시작하면 목록은 다시 1개다.

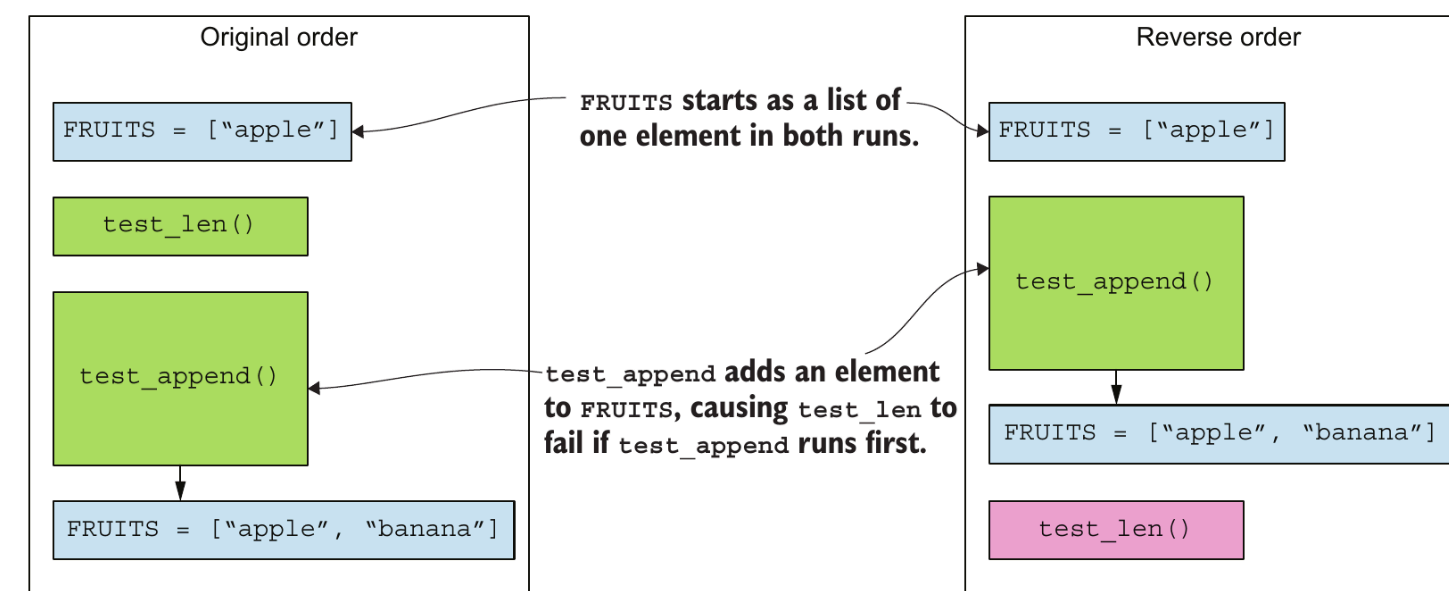


Figure 5.5 Tests that depend on state created by other tests may fail when reordered or moved.

추가 → 길이: 1 failed, 1 passed
길이 → 추가: 2 passed (새 실행)

09

fixture로 테스트마다 새 목록 준비

tests/test_state.py를 이 코드로 교체

- fixture는 테스트에 필요한 값을 준비하는 함수다. 전역 목록 대신 매번 새 목록을 반환한다.

```
import pytest

@pytest.fixture
def fruits():
    return ["apple"]

def test_append(fruits):
    fruits.append("banana")
    assert fruits == ["apple", "banana"]

def test_len(fruits):
    assert len(fruits) == 1
```

등록 → 요청 → 값 전달

- `import pytest`로 도구를 가져오고 `@pytest.fixture`로 바로 아래 준비 함수를 등록한다.
- 테스트 인자 `fruits`와 같은 이름의 함수를 pytest가 실행한다. 반환한 목록이 그 인자로 전달된다.
- 기본 설정은 테스트마다 준비한다. 이 함수 안의 `["apple"]`이 새 목록이므로 앞 테스트의 추가 내용이 남지 않는다.

```
python -m pytest tests/test_state.py -q
```

```
test_append: ["apple"] → ["apple", "banana"]
test_len:     새 ["apple"] → 길이 1
2 passed
```

순서를 반대로 지정해도 2 passed

10

parametrize로 입력 두 개 검사

tests/test_greeting.py를 이 코드로 교체

- 동일한 인사 검사를 복사하지 않고 입력·기대값의 두 행으로 표현한다. `greet`의 구현은 처음과 같다.

```
import pytest
from hello_open_source.greeting import greet

@pytest.mark.parametrize("name, expected", [
    ("Hee", "Hello, Hee!"),
    ("Kim", "Hello, Kim!"),
])
def test_greet(name, expected):
    assert greet(name) == expected
```

한 데이터 행마다 테스트 한 번

- `@pytest.mark.parametrize`는 아래 테스트를 데이터 행마다 실행한다. `"name, expected"`는 값을 받을 두 인자의 이름이다.
- 첫 행은 `name="Hee", expected="Hello, Hee!"`다. 둘째 행은 Kim의 인사말을 비교한다.
- fixture는 준비할 값을 제공하고, parametrize는 검사할 입력 사례를 늘린다. 이 예제는 fixture 없이 실행한다.

```
python -m pytest tests/test_greeting.py -q
```

```
Hee → "Hello, Hee!"와 비교: 통과
Kim → "Hello, Kim!"와 비교: 통과
2 passed
```

coverage: 코드·실행 명령·결과

별도 coverage_demo/ 폴더의 파일 두 개

`greeting.py`: 빈 문자열이면 기본 인사말을 반환하는 독립 예제다. 기존 패키지는 바꾸지 않는다.

```
def greet(name):
    if name == "":
        return "Hello, world!"
    return f"Hello, {name}!"
```

`test_greeting.py`: 이름 입력만 검사한다.

```
from greeting import greet

def test_name():
    assert greet("Hee") == "Hello, Hee!"
```

```
cd coverage_demo
python -m pip install pytest pytest-cov
python -m pytest test_greeting.py -q \
    --cov=greeting --cov-report=term-missing
```

테스트 통과와 줄 실행 비율

- coverage는 실행된 코드의 비율이다. `pytest-cov`는 pytest에 측정을 더하는 확장 도구다. Python에서 import하지 않고 위 설치·실행 명령으로 사용한다.
- `--cov=greeting`은 greeting 모듈을 측정한다. `--cov-report=term-missing`은 미실행 줄 번호를 터미널에 표시한다.

Name	Stmts	Miss	Cover	Missing
greeting.py	4	1	75%	3

TOTAL	4	1	75%	
1 passed				

이름 검사는 통과했지만 빈 이름을 처리하는 3번 줄은 실행하지 않았다.

`.coverage`에 측정값을 기록하며, 같은 명령의 재실행은 그 값을 교체한다.

coverage 보고서의 항목과 해석

greeting.py의 줄 번호와 측정 결과

```
1 def greet(name):
2     if name == "":
3         return "Hello, world!"
4     return f"Hello, {name}!"
```

항목	이 예제에서의 뜻
Stmts 4	측정하는 실행 가능 줄 4개
Miss 1	실행하지 않은 줄 1개
Cover 75%	$(4 - 1) \div 4 \times 100 = 75\%$
Missing 3	미실행 위치는 3번 줄

모듈을 import할 때 1번 줄의 함수 정의도 실행된다. 빈 줄·주석은 세지 않는다. 여기서는 줄만 측정한다.

빈 이름 테스트 추가 후 같은 명령 실행

coverage_demo/test_greeting.py에 아래 테스트를 추가한다. ""는 길이가 0인 문자열이다.

```
def test_empty():
    assert greet("") == "Hello, world!"
```

Name	Stmts	Miss	Cover	Missing
greeting.py	4	0	100%	
TOTAL	4	0	100%	
2 passed				

- greet("")가 3번 줄을 실행해 4개 줄을 모두 실행했다. TOTAL은 측정 대상 파일 전체의 합계이며 여기서는 파일 1개다.
- 100%는 모든 줄을 지나갔다는 뜻이다. 반환값의 정확성은 assert의 기대값과 비교해야 확인된다.

13

공백 입력의 기대값과 coverage의 한계

교재 Figure 5.2: 입력에 따른 경로

- 파란 Code 상자는 코드다. 초록 점선은 흔한 정상 경로, 자홍 점선은 경계·오류 경로다. 화살표는 실행의 진행 방향이다.
- Edge case는 경계 사례, Error handling은 오류 처리다. 아래 예제의 빈 이름은 허용한 정상 입력이다.

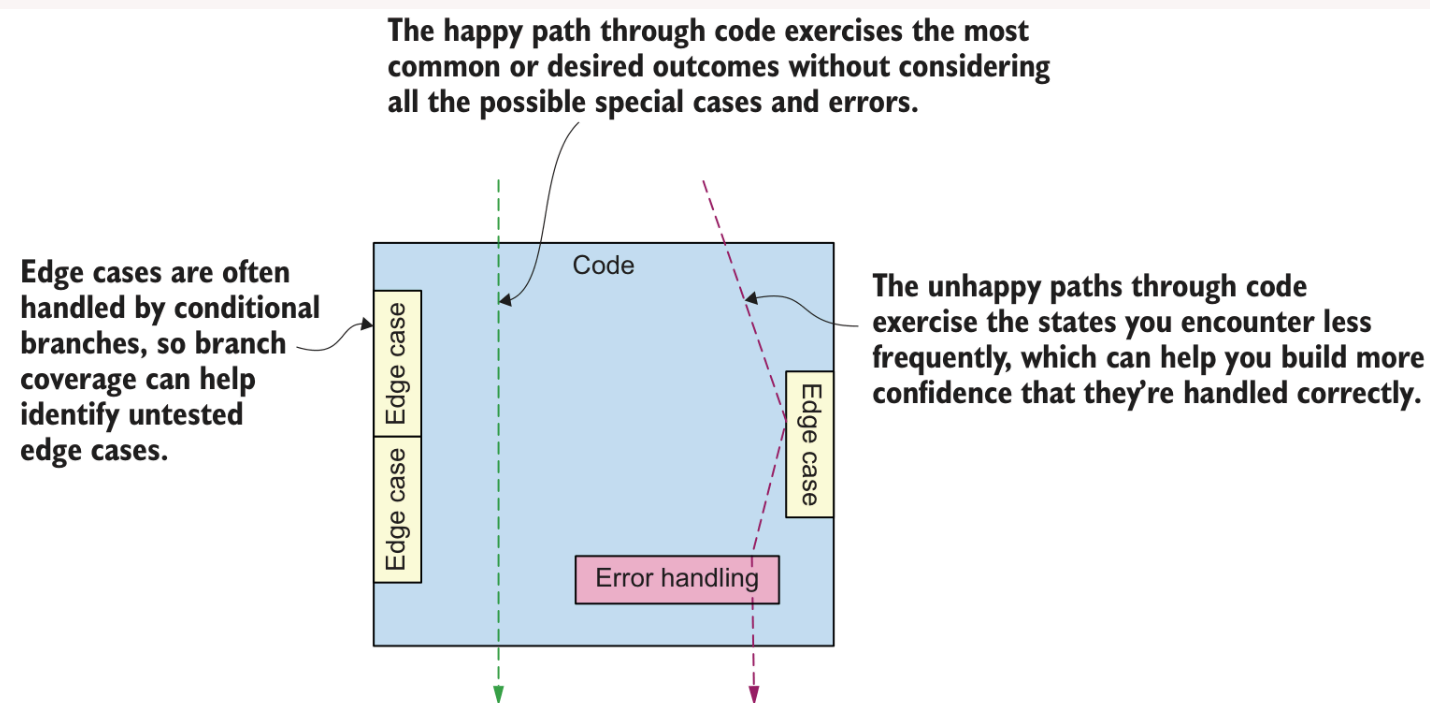


Figure 5.2 Tests may cover the common, desired execution paths or the less common edge and error cases.

공백 한 칸도 기본 인사로 처리할 것인가?

- `coverage_demo/greeting.py`는 `""`만 검사한다. `" "`는 공백이 한 칸 있어 다른 입력이다.
- 요구사항을 “공백만 입력해도 기본 인사말을 반환한다”로 정하면, 현재 구현은 이 요구를 만족하지 않는다.

```
greet("") → "Hello, world!" # 기존 검사 통과
greet(" ") → "Hello, !"    # 현재 구현
```

`test_greeting.py`에 추가할 검사

```
def test_space():
    assert greet(" ") == "Hello, world!"
```

이 검사를 추가하면 **1 failed, 2 passed**다. 줄 coverage는 여전히 **100%**지만 공백 처리 요구에는 실패한다. 입력별 기대값을 먼저 정해야 오류를 찾을 수 있다.

14

Summary

실행 결과에서 판단할 수 있는 것

- `-e .`로 설치한 기존 Python 소스를 고치면 새 실행에서 변경을 읽는다. pytest는 이름 규칙으로 테스트를 찾아 단언의 참·거짓을 보고한다.
- fixture가 테스트마다 새 목록을 준비하면 순서에 따른 간섭을 막는다. parametrize는 입력·기대값 두 행을 두 사례로 실행한다.
- coverage 75%는 미실행 3번 줄을 가리켰다. 빈 이름 테스트를 더해 100%가 되어도 공백 입력의 정확성은 별도 단언으로 확인해야 한다.